

Obfuscating Provenance-Based Forensic Investigations with Mapping System Meta-Behavior

Anyuan Sang
Xidian University
Xi'an, China
anyuan@stu.xidian.edu.cn

Yuchen Wang
Xidian University
Xi'an, China
ycwang24@foxmail.com

Li Yang*
Xidian University, Shaanxi Key
Laboratory of Network and System
Security
Xi'an, China
yangli@xidian.edu.cn

Junbo Jia
Xidian University
Xi'an, China
jbjia@stu.xidian.edu.cn

Lu Zhou
Xidian University
Xi'an, China
zhoulu@xidian.edu.cn

ABSTRACT

The provenance graph technique has gained popularity for attack analysis, such as Advanced Persistent Threat (APT) attacks, by creating entity interaction graphs from host audit logs. While this method has shown promising analysis results and interpretability, its robustness against mimic attacks carried out by potentially skilled attackers has yet to be fully proven. Recent research has showcased adversarial methodologies targeting provenance-based Machine Learning (ML) detectors, leading to evasion attacks through the addition of corresponding nodes and edges to the feature space. However, these approaches face several challenges, including the difficulty in translating feature alterations into practical attack scenarios and limited applicability to other provenance graph-based detection schemes.

In this study, we propose the PROVENANCE-BASED ATTACK INVESTIGATION OBFUSCATION FRAMEWORK (PAIOF), which proposes a novel obfuscation attack for the existing provenance-based forensic investigation system and demonstrates it needs to be extended and upgraded. More specifically, by thoughtfully analyzing key indicators from three classic provenance-based investigation approaches to set obfuscation goals. We establish an end-to-end mapping relationship between system operational behaviors and provenance graphs and create obfuscation programs guided by obfuscation goals to generate real attack instances. Our experiments demonstrate that our obfuscation scheme significantly reduces both the recall and precision of current state-of-the-art schemes in the DARPA Transparent Computing (TC) dataset and in simulated real-world attack scenarios. Furthermore, the meta-behavioral modeling approach of our system ensures its applicability in real-world scenarios, while we also discuss potential defenses against such attacks.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

RAID 2024, September 30–October 02, 2024, Padua, Italy

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0959-3/24/09

<https://doi.org/10.1145/3678890.3678916>

CCS CONCEPTS

• Security and privacy → Intrusion detection systems.

KEYWORDS

Attack Investigation, Evasion Attack, Provenance Graph

ACM Reference Format:

Anyuan Sang, Yuchen Wang, Li Yang, Junbo Jia, and Lu Zhou. 2024. Obfuscating Provenance-Based Forensic Investigations with Mapping System Meta-Behavior. In *The 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024)*, September 30–October 02, 2024, Padua, Italy. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3678890.3678916>

1 INTRODUCTION

Recent cyber incidents [1] have shown that traditional static security solutions (file hash, signatures, etc.) are insufficient to counter skilled attackers using multiple stealthy attack techniques. To defend against the potential threat of such advanced attackers, new dynamic defense techniques have emerged, among which, provenance-based attack detection and investigation techniques have been shown to be effective in detecting Advanced Persistent Threats (APTs) [7, 9, 11, 16, 18, 22, 29, 38, 39, 54, 56, 58].

The provenance graph represents the interaction between system entities in the system process, including file addition, deletion and modification, network sending and receiving, and process calls. The rich semantic information contained within provenance graphs provides security analysts with the capability to more comprehensively and accurately investigate and trace the origins and pathways of cybersecurity incidents. A line of studies have investigated the use of provenance graphs for analyzing cyberattacks and conducting subsequent forensic investigations, leading to the development of various technical approaches, including knowledge-tag-based [18, 39, 58], priority-based [12, 16, 35], graph compression-based [28, 51], and machine learning (ML) based [7, 48, 52, 56] methods. The common purpose of these efforts is to identify sets of nodes in the provenance graph that are associated with malicious activities, and to further deduce the development process and lifecycle of the attack behaviors. These approaches effectively extract in-depth

information from provenance graphs and analyze the implicit relationships among the entities within the graph, thereby significantly advancing the investigation of cyberattacks.

However, after a comprehensive review of these attack analysis and investigation approaches, it becomes apparent that there is a lack of understanding within the community regarding the robustness of these investigation methods against evasion attacks [21, 29, 31, 34]. Only [13] and [40] have demonstrated evasion attacks specifically on ML-based investigation schemes by inserting deliberately constructed nodes and edges into provenance graphs to manipulate investigation results. The robustness of other types of attack investigation methods against evasion attacks has not been clearly assessed. What is worse, attack methods that merely modify nodes and edges in provenance graphs can bypass attack detection but lack research on implementing these modifications through actual system behaviors. This results in attacks that are theoretically possible but not practically applicable, rendering them ineffective in real-world scenarios.

This lack motivates us to explore the capability boundaries of existing attack investigation methods by introducing new evasion attacks, particularly ensuring that the attacks are practically feasible from end to end, establishing a clear link between system behaviors and changes in provenance graphs. This approach is intended to provide a more comprehensive understanding of the current attack investigation methods in terms of security and applicability.

To this end, we propose a novel Provenance-based Attack Investigation Obfuscation Framework (PAIOF) for misleading existing attack investigation methods. The PAIOF first analyzes key indicators for identifying malicious activities within provenance graphs used in knowledge-tag-based, priority-based, and graph-compression-based attack investigation methods. Based on these indicators, it strategically formulates evasion objectives to guide the construction of attacks, thereby aiming to minimize the likelihood of being detected. Concretely, for each category of attack investigation method, we summarize their key indicators for detecting attacks (in Section 3.2) and derive the essential prerequisites necessary to avoid detection, known as obfuscation goals (in Section 3.3). The specific obfuscation goals are listed as follows :

- For bypassing priority-based investigation methods [12, 16], the attack should generate benign events characterized by high temporal correlation, significant differences in nodes' in/out degrees, and rare occurrence frequencies. Events with these features can create a more "obvious" path in the provenance graph, effectively concealing the true malicious activities and obfuscating the investigation process.
- For evading knowledge tag-based investigation methods [19, 39], the attack should generate meaningless actions that nonetheless satisfy the knowledge rules defined in the detection methods. This is reflected in the provenance graph through the addition of numerous fake but rules-compliant subgraphs. This strategy creates additional redundant information, which helps to obscure the true malicious activities.
- For obfuscating graph-compression based investigation methods [28, 46], the attack should introduce nodes relevant to the Point of Interest (POI) nodes, thereby injecting more noise into

the provenance graph to increase false positives and reduce the likelihood of the malicious nodes being detected.

The core challenge lies in realizing these obfuscation goals through actual system behaviors, ensuring that the constructed attacks are practical and effective from end to end. To address this, we model an end-to-end mapping between basic system operations and the provenance graph. Under the guidance of this mapping model and the proposed obfuscation goals, the necessary attack characteristics can be constructed by combining different system operations. It ensures that any operations on the provenance graph during the evasion attack are executed through actual basic system operations and ultimately encapsulated by functional programs, significantly enhancing the usability and applicability of evasion attacks in real-world scenarios.

We systematically evaluate the effectiveness of our proposed PAIOF. Specifically, we analyze the impact of our conducted evasion attacks on five state-of-the-art attack investigation methods (**LogGC** [28], **CPR** [51], **NoDoze** [16], **HOLMES** [39], **DE-PIMPACT** [12]) across four real-world datasets (DARPA THEIA, CADETS, TRACE and simulated datasets). By deploying attacks, we observe a clear degradation in detection and investigation performance under three metrics of all five investigation methods. Furthermore, we assess the applicability of PAIOF in real-world scenarios. Simulated attacks in a black-box environment demonstrate that PAIOF could achieve results comparable to those of white-box attacks under defense-agnostic conditions. Performance comparison results indicate that our evasion attacks increase the time and effort required for defenders to conduct attack investigations, while the computational costs associated with deploying PAIOF are acceptable. The contributions are summarized as follows:

- We are the first to present an end-to-end practical applicable obfuscation attack on existing provenance graph-based investigation methods. It enhances the understanding of the robustness of existing attack investigation approaches against evasion attacks and bridges the gap in the applicability of current evasion attack strategies.
- We model the mapping relationship between system meta-behaviors and their corresponding embeddings in provenance graphs, illustrating how an attacker could manipulate the result of attack investigation by obfuscating the sequence of program execution.
- We comprehensively evaluate the performance of our proposed attacks on five state-of-the-art attack investigation schemes in various scenarios. The experimental results show that PAIOF significantly reduces (up to 91%, with an average of over 60%) the precision rate of existing automated investigation methods and is able to generate effective evasion attacks in real-world scenarios.

2 BACKGROUND

2.1 Causality Analysis via System Audit

King et al. [23] initially develop a provenance graph using system audit logs to track the cause and effect of security incidents, which is essential for attack investigation and forensic analysis. System audit events document the interactions among system entities such

as processes, files, and network entities. The event typically revolves around the process as the subject, with the network and file as the objects. These interactions are depicted through system calls. For instance, *fork* signifies the relationship between a child process and its parent process. Fig. 1 demonstrates a causal analysis using the provenance graph. When presented with an alarm point (often generated by IDS or anomaly detection systems), forward analysis can identify subsequent related entities to understand the attack impact, while backward analysis can trace ancestor entities to pinpoint the entry point of the attack.

2.2 Provenance Graph

The provenance graph, defined as a directed graph that describes the calling relationship between system entities, has been widely used in various forensic investigations and analyses [7, 16, 18, 34, 38]. The basic element of the provenance graph is usually extracted from the audit log, which is represented as (*subject, object, relation*). The *subject* and *object* represent the main body of the system call. The *relation* represents the type of system call. Typically, the *subject* = {*process*}, *object* = {*file, socket*}, and the *relation* = {*fork, write, read, sendmsg, ...*}. Take a system audit event as an example, the *fork* can create an edge link from process *process₁* to process *process₂*, the *read* can create an edge link from process *process₁* to file *file₁*.

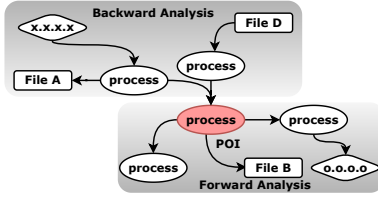


Figure 1: Illustration of causal analysis on provenance graph. Forward analysis extracts subsequent entities relevant to an alarm point (red node), while backward analysis traces its ancestor entities.

2.3 Threat Model

Attacker knowledge. In our experiments, PAIOF utilizes both the black-box model and the white-box model. In the black-box model, we assume that the attacker does not know the type of investigation system. Our scheme allows attackers to launch an attack without having to guess the type of the investigation system. On the other hand, in the white-box model, we allow attackers to execute specific model queries to infer the type of investigation system. For example, before carrying out the actual attack, the attacker can create low-frequency obfuscation programs that meet the investigation system’s indicator to perform model queries. This method does not impact the actual attack, as existing attack investigation systems are generally susceptible to the issue of ‘alert fatigue’ [12, 16, 35] leading to the neglect of low-frequency alerts. Additionally, attackers can use proxy hosts to execute model queries, thereby ensuring that the actual attack remains unaffected by the model queries. We also assume that the attacker is already familiar with the key nodes

of the Point of Interest (POI) event in both of two type models. This assumption is reasonable since the attack is carried out by the attacker. Therefore, the attacker can introduce more valuable events during the attack, including information leakage and internal investigation, which are usually flagged as POI events.

Investigation assumptions. In our experiment environment, we rely on a Trusted Computing Base (TCB) comprising an operating system, an auditing framework, and provenance analysis tools. It is assumed that all components in the TCB are fully functional from installation to completion, which is also standard among existing provenance-based detectors. Therefore, we do not consider the hardware trojans and side-channel attacks that cannot be captured by audit in this paper. We also operate under the assumption that the attacker does not have prior knowledge of the system being tested. Additionally, we assume the integrity of output audit data be ensured by existing secure provenance systems [8, 42–44] and integrity audit systems [26, 28, 36, 37, 55].

Execute assumptions. PAIOF uses some of the external programs to implement some of the functionality, specifically, PAIOF relies on external programs like *ifconfig* and *request* to create obfuscation programs, which depend on system calls such as *socket()*, *send()*, and *recv()*. Therefore, it is essential to assume that the required system calls for executing obfuscation programs are present on the target hosts, which is common on hosts running Linux or Windows operating systems.

3 PAIOF OVERVIEW

To understand the robustness of provenance-based attack investigation methods against complicated attacks, we intend to take the analysis of the key indicators for judging attacks in the existing investigation methods as the foothold. Subsequently, we create more ‘obvious’ attack subgraphs in provenance graphs under the guidance of the analysis of these key indicators.

Concretely, we conduct a thorough analysis of typical attack investigation methods [12, 16, 28, 39, 51], building upon the taxonomy proposed in [10, 21, 30]. The analysis primarily focuses on identifying the key indicators used by these investigation methods to determine attacks. Based on these key indicators, we abstract and deduce the fundamental prerequisites required to evade detection by attack investigation mechanisms, i.e. obfuscation goals. To ensure the effectiveness of the attack programs while achieving these obfuscation goals, we establish an end-to-end mapping relationship between system operational behaviors and provenance graphs and create obfuscation programs guided by obfuscation goals to generate real attack instances. This approach enables us to circumvent the detection of attack investigation methods, and simultaneously facilitate reliable attacks without compromising the functionality of the attack programs. Figure 2 illustrates the workflow of our attack.

3.1 Attack investigation methods

In accordance with the taxonomy of attack investigation proposed in [10, 21, 30], we focus our analysis on three principal categories: priority-based, knowledge-tag-based, and graph-compression-based attack investigations. The following section

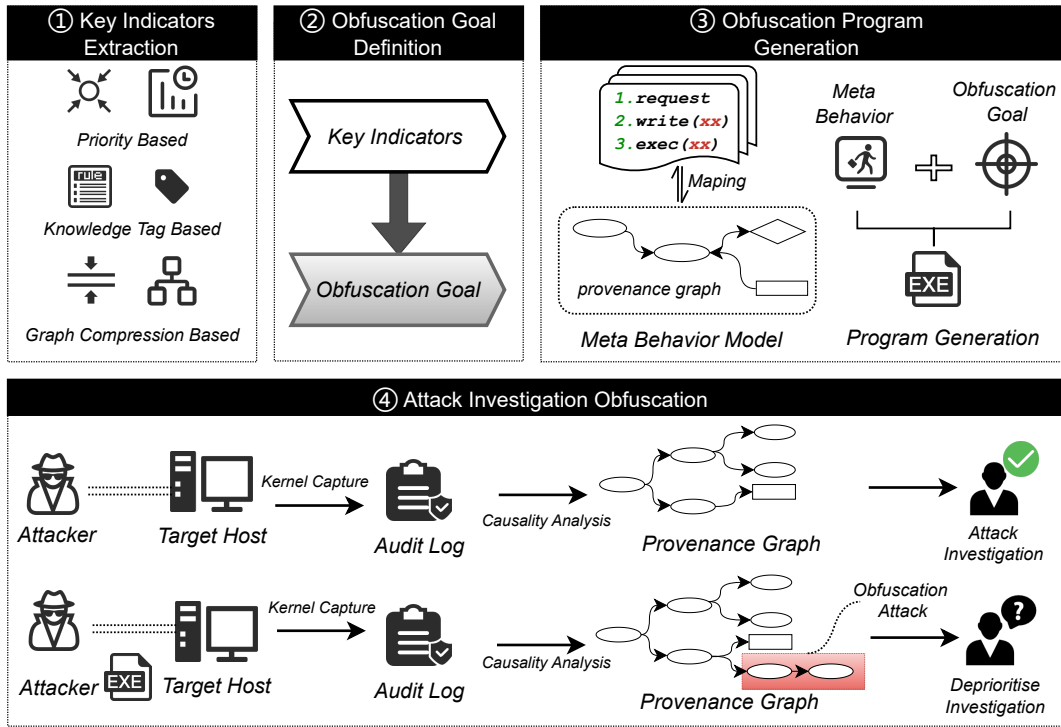


Figure 2: The overall illustration of the PAIOF scheme. (1) Key Indicators Extraction (Section 3.2): Analyzing key indicators of investigation approaches. (2) Obfuscation Goal Definition (Section 3.3): Constructing obfuscation goal based on investigation method key indicators. (3) Obfuscation Program Generation (Section 3.4): Modeling meta-behavior to automate obfuscation program construction. (4) Attack Investigation Obfuscation: Using obfuscation programs to insert the more ‘obvious’ subgraph in the provenance graph, deprioritize attack investigation priority.

provides detailed explanations of the workflows and characteristics associated with each of these attack investigation approaches.

- **Priority-based attack investigation.** This category of attack investigation methods typically leverages node attributes, such as frequency and fanout number, to calculate and propagate threat level indicators across the dependency graph. It utilizes alarm information from existing Intrusion Detection Systems (IDS) as the starting node for initiating the attack investigation. This approach prioritizes these threat indicators to perform a backward search, aiming to uncover the most significant threat-inducing attack chain. Within this context, we explore two distinct priority-based approaches: the whole path priority-based scheme NoDoze [16], and the node priority-based scheme DEPIMPACT [12].
- **Knowledge-tag-based attack investigation.** This category of methods generally matches attack patterns in provenance graphs based on manually summarized matching rules such as Tactics, Techniques, and Procedures (TTPs), which relies on domain knowledge in cyberattack modeling and analysis. The subgraphs that match the rules are identified to form an attack graph, and a set of matched threat nodes are outlined as a possible attack chain. We consider the typical attack investigation methods HOLMES [39] as the benchmark since

they provide comprehensive knowledge rules for practical implementation.

- **Graph compression-based attack investigation.** Graph compression is a common technique used to improve the efficiency of attack analysis. Some strategies focus on reducing the size of the graph by decreasing the number of nodes. These strategies argue that cache nodes and intermediate variables in program execution have minimal impact on the investigation and can be removed from the provenance graph. Besides, a line of studies focuses on compressing edges, targeting the reduction of redundant IO and network interactions that typically occur in large volumes at program startup. By applying these compression techniques, the resulting dependency graph is smaller and more manageable, facilitating easier analysis and quicker response by analysts. We consider two typical works CPR[51] and LogGC[28] as the victim model and expect to circumvent these provenance-based investigation methods.

3.2 Key indicator extraction

To obfuscate existing attack investigation schemes, we analyze each of the three types of attack investigation schemes and summarize

their key investigation indicators. These indicators will serve as the basis for creating obfuscation programs.

3.2.1 Key indicators for priority-based investigation. In the node priority-based attack investigation scheme DEPIMPACT, three types of features are extracted for each edge to compute the dependency weights. The threat weights are then determined based on these features. The threat weights are in the form of:

$$W_e = \omega_S * F_S(e) + \omega_T * F_T(e) + \omega_C * F_C(e), \quad (1)$$

where W_e represents the weight of event e , $F_S(e)$ represents the data flow correlation of edge e with POI events, and $F_T(e)$ represents temporal correlation, i.e. edges appearing at relatively the same time are more likely to be correlated. The $F_C(e)$ represents the concentration ratio, i.e. the ratio of the number of outgoing edges to the number of incoming edges, which gives higher weight to nodes that are reachable from more than one backward path. The ω_* represents the weights of the three types of the weights of features, the author uses a discriminative feature projection scheme based on Linear Discriminant Analysis (LDA) to compute the projection vectors.

In the path-priority-based attack investigation scheme NoDoze, the calculation of the anomaly score for the entire path involves three main components: event frequency, event out-degree score, and event in-degree score. The specific formula is as follows:

$$RS(Path) = \prod_{i=1}^l IN(Source_i) \times M(\epsilon_i) \times OUT(Destination_i), \quad (2)$$

$$AS(Path) = 1 - RS(Path), \quad (3)$$

here, $RS(Path)$ represents the regulatory score, which is required to calculate the node frequency, out-degree scores, and in-degree scores on the overall event path as described in NoDoze. The $OUT(Destination_i)$ and $IN(Source_i)$ represent the stability of output and input data flow. The $M(\epsilon_i)$ represents the likelihood of occurrence, based on historical frequency compared to all events of the same type at the starting node. If an event has never occurred, its likelihood is 0. The $AS(Path)$ represents anomaly scores, derived by subtracting the regulatory score from 1.

The above analysis demonstrates that priority-based schemes rely on the node's graph structure attributes, rarity, and other statistical information to quantify the node's abnormality and support priority analysis. Therefore, the key indicator of priority-based methods is the node's attribute parameter.

3.2.2 Key indicators for knowledge tag-based investigation. In the knowledge tag-based investigation scheme, HOLMES establishes a series of Tactics, Techniques, and Procedures (TTPs) based on the APT lifecycle model [4] to depict the APT status of the node. This system conducts real-time incremental matching within a memory-based provenance graph. When the event's subject, object, and type meet the predefined criteria in the rules, the nodes' states in the provenance graph are updated. Subsequently, the High-level Scenario Graph (HSG) is derived by extracting nodes and edges that adhere to the rules. The process of conducting an attack investigation in this HSG is to first identify a set of events paths that match all the process TTPs to form a threat tuple $\langle E_1, E_2, E_3, E_4, E_5, E_6 \rangle$.

Each TTP's threat level is evaluated using the Common Vulnerability Scoring System (CVSS) [2], and they are ranked accordingly using a specific equation:

$$\prod_{i=1}^n E_i > \sigma, \quad (4)$$

here n is the number of APT states, E_i is the event anomaly score, and σ is the detection threshold.

Knowledge tag-based approaches often rely on strict rules for matching and analyzing nodes according to these rules. The key indicator of this approach is its adherence to predefined knowledge rules.

3.2.3 Key indicators for graph compression-based investigation. Current attack investigation schemes utilize graph compression techniques, such as LogGC and CPR, to enhance the indirect dependency graph by compressing edges and nodes. Through backward analysis of the compressed graph, the attack path can be identified. The key indicator of these schemes is to achieve higher compression ratios while maintaining effective attack investigation.

3.3 Obfuscation goal definition

To effectively develop obfuscation programs that can impede provenance-based investigation approaches and complicate analysts' assessments, it is crucial to define specific obfuscation goals for different scenarios. These scenarios encompass three types of provenance-based investigation techniques: priority-based, knowledge tag-based, and graph compression-based. Moreover, the establishment of obfuscation goals for each technique is guided by key indicators identified through feature-level analysis in the previous section 3.2. In addition, in order to better derive obfuscation goals, a goal paradigm is established to guide the definition of specific obfuscation goals. The overarching goal Tar is to maximize the false positive rate (FPR) of the compromised investigation system, it is set as follows:

$$Tar = \max \left(\frac{FP(G + \Delta G_{sub})}{FP(G + \Delta G_{sub}) + TN(G + \Delta G_{sub})} \right) \quad (5)$$

where G represents the original provenance graph, ΔG_{sub} represents the additional subgraphs. FP and TN denote the false positives and true negatives, respectively. By adding these subgraphs, we aim to increase the false positive rate, thereby complicating the investigation process and obfuscating the true attack paths.

3.3.1 Priority-based method. Priority-based schemes evaluate the structural properties of graphs in relation to nodes and edges to assess their respective threat scores. To obfuscate these schemes, we establish the obfuscation goal for the priority-based method $Tar(P_1)$ as:

$$Tar(P_1) = G.insert(combine(F_e, T_e, D_e)), \quad (6)$$

where G is the provenance graph. F_e denotes the frequency score of the event. T_e and D_e are the temporal relevance score and the access balance score, respectively. The idea behind us is that considering the impact of graph node attributes on investigation priority, we try to create paths with more significant impact as a way to lower the priority of the real attack paths.

3.3.2 Knowledge tag-based method. Knowledge tag-based methods are summarized based on existing attack patterns, making conducting a zero-day attack without any existing rule coverage prohibitively expensive for the attacker. While this rule-based approach ensures strict matching in the provenance graph, it also opens up opportunities for attackers to abuse the rules by creating a high volume of false alarms, potentially diverting attention from real attack paths. Based on this analysis, our obfuscation goal for the knowledge tag-based method $Tar(P_2)$ is set as follows:

$$Tar(P_2) = G.insert(G_m), \quad (7)$$

where G is the provenance graph, and G_m is a subgraph that matches the knowledge rules. The significance of setting this goal is to include fake but matching rule-compliant attack subgraphs in the provenance graphs that appear legitimate according to predefined rules and criteria.

3.3.3 Graph compression-based method. Compression schemes initially compress the provenance graph before conducting forensic investigation centered around a specific POI node, utilizing all linked nodes as potential paths. Despite the removal of numerous redundant edges through the compressed graph technique, the approach is still prone to a significant number of false positives. Therefore, we set the obfuscation goal for the knowledge graph compression-based method $Tar(P_3)$ as:

$$Tar(P_3) = G.insert(n * G_{rel}(poi)), \quad (8)$$

where G is the provenance graph, $n * G_{rel}(poi)$ is the number n of obfuscated subgraphs associated with POI events. The significance of this goal is to add enough obfuscated subgraphs associated with POI nodes to the provenance graphs.

Our scheme differs from recent ML-based evasion approaches [13, 40] by focusing on existing forensic investigation methods. Unlike previous schemes that concentrate on altering the structural characteristics of nodes near the attack, our approach involves incorporating obfuscated subgraphs created by the obfuscation program into the attack chain, which enhances the practical implementability of our programs.

3.4 Obfuscated program generation

Depending on the obfuscation goals summarized for multiple investigation schemes, we can construct obfuscation attacks. To create practical applicable and usable obfuscation programs, we model an end-to-end mapping between system operational behavior and provenance graphs. Specifically, we initially model the mapping relationship between the basic system operation behaviors and the nodes and edges in the provenance graph. Subsequently, using the obfuscation goal as a guide, we combine the basic system operations to create system meta-behaviors that can satisfy the obfuscation goal and thus lower the priority of the investigation system. Lastly, we develop an automated obfuscation program that dynamically combines the system meta-behaviors according to the type of investigation system and packages them into a practical obfuscation program.

3.4.1 Basic behaviour mapping. In order to guarantee the practical applicability and executability of obfuscated programs, we have

created a mapping relationship between the fundamental system operational behavior and the provenance graph. This approach ensures that the system's operational behavior not only maintains executability but also facilitates the construction of programs that meet obfuscation goals.

Table 1: Basic Behavior Map.

System Behavior (function)	Description	Provenance Graph (edge)
read(f), open(f)	program reads file or opens file	$p \leftarrow f$
write(f)	program writes file	$p \rightarrow f$
exec(p_1), subprocess(p_1)	program forks the new process	$p \rightarrow p_1$
post(s), get(s), request(s)	program makes a request	$p \rightarrow s$
recv(s), recvfrom(s)	program receives message from socket	$p \leftarrow s$

To implement this idea, we utilize functions in a well-known high-level language (e.g., Python and C++) to document the changes in the provenance graph, generating a mapping relationships Table 1. The relationships between system operations such as processes to files, processes to processes, and processes to networks are established as edges and nodes in the provenance graph. For instance, the fundamental system operation $open(f)$ is represented in the provenance graph as a process p pointing to an edge of the type 'open' of a file f .

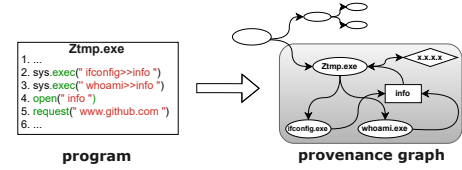


Figure 3: Illustration of meta-behavioral modeling, where the program part on the left produces the corresponding edges and nodes (grey part) in the provenance graph after execution.

3.4.2 Model fundamental meta-behaviour. To satisfy the obfuscation goals elements (F_e , T_e , D_e , G_m , $G_{rel}(poi)$), we dynamically combine the sequences of basic system operations to achieve this, and refer to the sequences of operations that satisfy the obfuscation goals as "meta-behaviors". Figure 3 provides an example of meta-behavior when attackers gain access to the target host, the information-gathering programs they use or commands they execute result in the creation of corresponding edges and nodes in the provenance graph. The meta-behaviors are detailed as follows.

In-out-degree difference. The difference between the in-degree and out-degree of a node can indicate its anomaly level in a priority-based investigation approach. This difference is a crucial factor in attack investigation, as highlighted in our analysis of the obfuscation goal D_e . To enhance this indicator, we aim to amplify the in-out-degree difference through a meta-behavior strategy. In Figure 4, we illustrate this by establishing connections from the *obf.exe* node to multiple socket nodes using the *request* procedure call iteratively, and linking the *info* node to the *obf.exe* node through an *open* procedure call. This approach allows us to

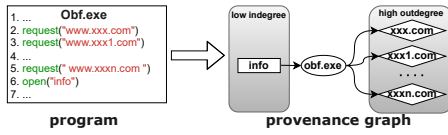


Figure 4: Example of a process node with higher access, with more send request calls and fewer read calls in the *obf.exe* program.

identify *obf.exe* nodes and their subgraphs with significant in-out-degree disparities, generating misleading and prominent obfuscated subgraphs within the priority-based attack investigation method.

High temporal correlation. The temporal correlation between timestamps of POI events is crucial in investigations. Investigators often prioritize critical nodes, such as the data leak node, at the beginning of their investigations. This priority is also reflected in the obfuscation goals T_e and $G_{rel(poi)}$. To model the obfuscation program, fake POI events are artificially introduced to increase temporal correlation. For example, in Figure 5, we demonstrate this concept. Specifically, information-gathering operations were conducted through the *ztmp.exe* node by executing *ipconfig* and *whoami* using the *exec* function. Subsequently, messages were sent to *github.com* using *request*, triggering data leakage rules. As a result, the node would often be identified as a POI and prompt an attack investigation. Attackers leverage this technique to create fake sequences with high temporal correlation to obfuscate the investigation system.

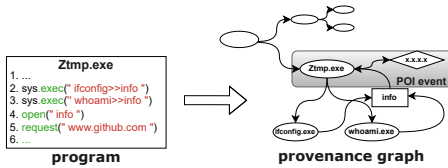


Figure 5: Illustration of temporal correlation on provenance graph. *Ztmp.exe* forges POI events and the events are highly correlated with other events.

Rareness process nodes. Rareness can indicate how often a node appears in a previous time segment, serving as a crucial factor in prioritizing attack investigation strategies. A higher rarity suggests a higher chance of the node being an attacking node, which aligns with the criteria established by the obfuscation goal F_e . By introducing rarer nodes into the graph, we aim to increase the probability of them being misclassified as attack nodes. In Figure 6, we present a specific scenario where we have successfully induced a misclassification in the attack investigation system. We execute the rarity process *rare.exe* with an *exec* function to introduce more low-frequency nodes in the provenance graph.

In the practical deployment, it is important that the attacker can manipulate the 'rareness' feature by using more intricate and less commonly repeated process names without needing specific knowledge of rare processes on the target host. This is because the 'rareness' feature is generally derived from audit logs collected over a period of time [16, 40]. By traversing these audit logs, key

(process name)-value (count) pairs are stored, and the inverse of the value is used as the node's rareness. Using more intricate and less commonly repeated process names could ensure that the process node receives higher rareness scores in the investigation system.

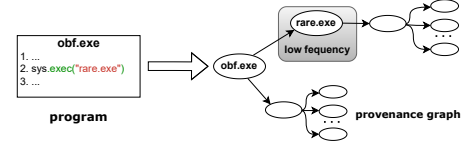


Figure 6: Illustration of using rarer process nodes on provenance graph. *obf.exe* acts as an intermediate node by executing rarer processes.

Meeting the knowledge rule. In the knowledge tag-based attack investigation system, adherence to knowledge rules is crucial. This entails ensuring that nodes comply with the constraints of established knowledge rules. This requirement is also reflected in the obfuscation goal G_m . To achieve this, we aim to introduce a set of nodes and edges that meet the rules. Illustrated in Figure 7, we execute sensitive operations such as *whoami* and *ipconfig* using *ztmp.exe*, thereby triggering the information gathering rule. Subsequently, we simulate a fake request using *request*, triggering the data leak rule. By performing these actions, we can generate subgraphs that are deceptive yet compliant with the knowledge rules, ultimately obfuscating the system's result. Note that PAIOF relies on more commonly used knowledge rules from public models that are easily covered by rule-based investigation systems, rather than obtaining them from specific investigation systems (e.g., HOLMES [3]). These knowledge rules are defined based on existing APT models (MITRE ATT&CK, Mandiant APT lifecycle), which are publicly available and provide rule implementations.

Although some of our meta-behaviors, such as high temporal correlation and meeting the knowledge rule, may appear to be part of the actual attack process in the provenance graph (collect information -> send to unknown socket), this is simply accomplished by utilizing the execution of certain external programs, such as *request*, in our system execution sequence. In fact, these obfuscation programs are designed to be executed concurrently with the attack, generating more convincing subgraphs to conceal the actual attack process. These obfuscation programs must be delivered to the target host and executed by the attacker through communication channels (e.g., C&C connections) during the attack sequence.

3.4.3 Obfuscation program generation. In the preceding section 3.3 and section 3.4.2, we outlined the obfuscation goal for different types of attack investigation schemes and constructed a system meta-behavior model to attain these goals. By integrating these two components, we can develop a practical and effective obfuscation program.

To automate the creation of obfuscation programs based on meta-behaviors, we employ algorithm 1. Initially, we provide the obfuscation goal X and the threat model T as inputs. Subsequently, we dynamically combine sequences of meta-behaviors. Finally, we utilize a program packaging tool (e.g., pyinstaller) to compile it into an executable program for real-world deployment.

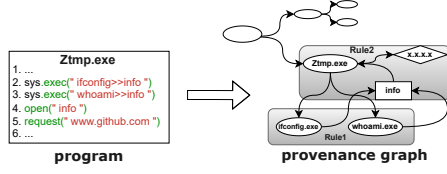


Figure 7: Ztmp.exe triggers the rules by executing a sequence of processes that match the existing rules. Fake nodes and edges are generated in the provenance graph.

Algorithm 1 Obfuscation Program Generation

Require: Obfuscation Goal List X and Threat Model T ▷ X
includes $Tar(P_*)$

- 1: $D \leftarrow$ Meta-behavior of degree difference ratio
- 2: $H \leftarrow$ Meta-behavior of high temporal correlation
- 3: $F \leftarrow$ Meta-behavior of rarer process node
- 4: $O \leftarrow$ Meta-behavior of meet exists rules
- 5: $Sequences \leftarrow \emptyset$
- 6: **for** $tar \in X$ **do**
- 7: **if** T is black-box model **then**
- 8: $Sequences \leftarrow \text{COMBINEAVERAGE}(U \cup D \cup H \cup F)$
- 9: **else if** tar is priority-based **then**
- 10: $Sequences \leftarrow \text{COMBINE}(D \cap H \cap F)$
- 11: **else if** tar is knowledge tag-based **then**
- 12: $Sequences \leftarrow \text{COMBINE}(O)$
- 13: **else if** tar is graph-compression-based **then**
- 14: $Sequences \leftarrow \text{COMBINE}(O \cup D \cup H \cup F)$
- 15: **end if**
- 16: **end for**
- 17: **return** $\text{SEQ2EXE}(Sequences)$ ▷ Program packing tools

Unlike the current method of directly adding nodes to the dependency graph [13, 40], our approach offers the attacker the end-to-end ability to utilize obfuscation techniques before carrying out the attack. This helps decrease the chances of the true attack path being identified. Furthermore, our construction of end-to-end system behavior and provenance graph mapping relationships can help us generate realistic and usable attack samples. In addition, our attack is extensible although we only list five meta-behaviors; with obfuscated goals, we can then build new meta-behaviors based on the mapping relationships in 3.4.1.

4 EVALUATION

Evaluation Methodology. We commenced by evaluating the obfuscation impact of PAIOF on the feature space (dataset) by directly incorporating subgraphs into the provenance graph, thereby assessing the efficacy of PAIOF. Subsequently, we conducted various real-world attacks and integrated our obfuscation program into the attack procedures to appraise the effectiveness of PAIOF in real-world attack scenarios. Following this, we executed our evaluation steps under a more stringent black-box framework to scrutinize the black-box capabilities of PAIOF. Finally, we investigated the influence of the quantity of incorporated obfuscation programs on both attacker and model capabilities.

Specifically, we evaluated three types of schemes: (1) priority-based: DEPIMPACT [12] and NoDoze [16], (2) knowledge-tag-based: HOLMES [39], (3) compressed graph-based: LogGC [28], CPR [51]. In addition, we validate the effectiveness of PAIOF on the feature space on the DARPA TC dataset [5] and deploy our obfuscation program in real-world attacks implemented by attack paradigms present in previous studies [12, 39, 51].

- **DEPIMPACT** [12]. The key parameters for conducting the attack investigation were defined based on three types of priority parameters outlined in DEPIMPACT: temporal correlation, stream correlation, and access degree concentration difference. Furthermore, we use the average contribution to measure the final prioritization metrics.
- **NoDoze** [16]. NoDoze executes profiles to measure path rarity and thus speed up attack investigation, so we use the daily log files of the deployed system as the execution profile and normal events from the DARPA TC data as the execution profile. In the attack investigation, we compute the anomaly scores across the graph and then sort the paths by anomaly scores to reduce the size of the graph.
- **HOLMES** [39]. HOLMES furnishes comprehensive TTP rules facilitating our implementation. Precisely, we implement these TTP rules according to the specifications outlined in the paper, enabling real-time matching with stream processing patterns. This allows for the confirmation and updating of node labeling statuses.
- **CPR** [51]. CPR merges edges generated by a large number of events occurring in a short period between two points in the provenance graph, specifically, we set up a 10-second time window and merge duplicate edges generated during the window time.
- **LogGC** [28]. LogGC removes nodes that were not relevant to the investigation of the attack, and we remove temporary files, library file nodes, and cache files in the provenance graph as required by LogGC.

Hyperparameter Setup. In our experimental scenario, we take the number of all inserted obfuscation programs (meta-behavior) n and add nodes k as the hyperparameter. The number of insertions of obfuscated subgraphs impacts the precision and recall of the attack investigation scenarios, which we discuss in the subsequent subsections.

Metrics. In order to evaluate the effectiveness of PAIOF, we set the recall and precision as metrics. Our experimental scenario changes the structure of the provenance graph, in other words, PAIOF adds more false positive nodes, so we choose precision, an FP-sensitive metric, to measure the effect of our scheme. In addition, we use FPR, F1, and Recall as evaluation metrics.

4.1 Evaluation Datasets.

DARPA TC dataset [5]. The DARPAT Transparent Computing Engagement 3 Data is widely used as a dataset for conducting provenance-based attack detection and analysis [14, 19, 34, 54, 56], which contains two weeks of audit logs originating from different hosts and containing multiple types of attack patterns, and we used TRACE, THEIA, and CADETS data in our experiments.

Table 2: Introduction and description of the attack scenarios used for experimental evaluation.

Attack case	Description of attack	Ref.
Firefox Backdoor	Firefox is utilized to perform memory loads, providing a remote console to export sensitive information.	TRACE case1 THEIA case1
Browser Extension	Using a malicious browser extension <i>pass_mgr</i> to attack the target host, the compromised plugin downloads and executes malicious programs for internal reconnaissance and leaks sensitive information.	TRACE case2 THEIA case2
Executable Attachment	A phishing email with an executable attachment that, when opened, establishes a connection to the attacker's machine.	TRACE case3
Phishing Email	The attacker sends a phishing email to the target host that contains a link to a malicious website used to collect user information.	THEIA case3
Nginx Backdoor	Attacker uses an Nginx in-memory exploit to drop several malicious executables, which are executed in SSH connections to exfiltrate files.	CADETS case
Phishing Email Executable Attachment	Attacker exploits Email phishing with an attachment containing malicious Excel spreadsheet macro, which executes a <i>PowerShell</i> session gathers system information and exfiltrates information.	Liu et al.[34]
Drive-by Download	The user visits a malicious website with a vulnerable <i>firefox</i> browser. As a result, a file named net is dropped and executed on the victim's host and then executes commands to collect information, finally exfiltrate information.	Milajerdi et al.[39]
ShellShock	An attacker utilizes an Apache server to trigger the Shellshock vulnerability in Bash multiple times.	Xu et al.[51]
Trojan	A user is convinced to download a malicious Trojan program (text editor) via <i>firefox</i> . Next, the user executes it leading to C&C channel being created. The attacker launches a shell prompt executes information-gathering commands and exfiltrates information.	Milajerdi et al.[39]
Wget-gcc	Malicious source files are downloaded and then compiled.	Xu et al.[51]
Backdoor Download	A user using a browser inadvertently downloads and unzips a backdoor program, which then links to the C&C server and collects system information resulting in exfiltration information.	Fang et al.[12]

Table 3: Effective of each technique in the dataset and PAIOF's performance in tech method.

Dataset	Graph-Compression				Knowledge-Tag				Priority							
	LogGC+CPR		LogGC+CPR (PAIOF)		HOLMES		HOLMES (PAIOF)		NoDoze		NoDoze (PAIOF)		DEPIMPACT		DEPIMPACT (PAIOF)	
	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.
TRACE Case1	1.00	>0.01	1.00	>0.01(-50.8%)	1.00	0.02	1.00	>0.01(-82.6%)	0.45	1.00	0.07(-.38)	0.33(-.66)	0.92	1.00	0.15(-.77)	0.66(-.33)
TRACE Case2	1.00	>0.01	1.00	>0.01(-50.8%)	1.00	0.06	1.00	>0.01(-82.6%)	0.33	1.00	0.11(-.22)	0.33(-.66)	1.00	0.69	0.33(-.66)	0.21(-.48)
TRACE Case3	1.00	>0.01	1.00	>0.01(-50.8%)	1.00	0.01	1.00	>0.01(-82.6%)	0.89	1.00	0.21(-.68)	0.25(-.75)	1.00	0.61	0.16(-.84)	0.21(-.40)
CADETS Case1	1.00	>0.01	1.00	>0.01(-32.2%)	1.00	0.01	1.00	>0.01(-67.3%)	0.30	1.00	0.1(-.2)	0.33(-.66)	0.90	1.00	0.3(-.60)	0.23(-.77)
THEIA Case1	1.00	>0.01	1.00	>0.01(-99.8%)	1.00	0.01	1.00	>0.01(-83.9%)	0.45	1.00	0.14(-.31)	0.2(-0.8)	1.00	0.01	0.42(-.58)	0.01(-)
THEIA Case2	1.00	>0.01	1.00	>0.01(-52.4%)	1.00	0.01	1.00	>0.01(-83.9%)	0.62	1.00	0.12(-.50)	0.33(-.66)	1.00	0.02	0.5(-.50)	0.01(-)
THEIA Case3	1.00	>0.01	1.00	>0.01(-62.2%)	1.00	0.12	1.00	>0.01(-83.9%)	0.33	1.00	0.11(-.22)	0.33(-.66)	1.00	0.33	0.10(-.87)	0.21(-.12)
Average	1.00	>0.01	1.00	>0.01(-87%)	1.00	0.03	1.00	>0.01(-81%)	0.48	1.00	0.12(-.36)	0.3(-.70)	0.90	0.52	0.28(-.62)	0.22(-.30)

Simulation Datasets. We deployed a red-blue adversarial environment and designed several attacks based on several attack paradigms covered in previous research [12, 34, 39, 51] in our lab environment, our specific environment is as follows.

- Attacker: We use the Kali Linux 64-bit operating system on the machine Intel(R) Core(TM) i9-10920x CPU (3.5GHz), 128GB RAM.
- Target Machine: We use the Windows 10 and Ubuntu 1804 operating system on the virtual machine Intel(R) Core(TM) i9-10920x CPU(3.5GHz), 128GB RAM.

In order to make our target host close to a real corporate environment, we conducted various service processes on our target host, such as mail systems and program upgrades. Additionally, the system had 5 active users to ensure that the audit logs captured contained sufficient redundant information unrelated to the attack. The target host was able to generate over 1 million audit logs per day. We simulated six previous typical attack cases for the attackers, including Phishing Email, Drive-by-download, Trojan, Wget-gcc, ShellShock, and Backdoor download. The attack scenarios used in our experiments are summarized in Table 2.

4.2 Attacks Performance in Feature Space

To assess the effectiveness of current schemes in the feature space, we constructed two scenarios: the black-box scenario and the white-box scenario. In the black-box scenario, we assume that the attack investigation scheme is unknown to the attacker, while the detection scheme in the white-box scenario is transparent to the attacker, allowing them to choose meta-behaviors based on the detector's preferences. Our analysis of the feature space in this section and the examination of real-world attacks in the following section are conducted in white-box scenarios. A more thorough evaluation in rigorous black-box scenarios will be elaborated on in the Section 4.4.

As our obfuscation program is tailored for real-world attack scenarios, certain adjustments are necessary within the feature space. Specifically, we execute the practical applicability obfuscation program to generate an attack subgraph, which is subsequently inserted into the existing provenance graph constructed from the DARPA TC dataset. For each method of attack investigation, we adopt the obfuscation goal delineated in Section 3.3.

To compare the impact of PAIOF's addition, we initially evaluated the efficacy of five current investigation methods in the DARPA TC

Table 4: Effective of each technique in real-world attack.

Attack Case	Graph-Compression				Knowledge-Tag				Priority-based							
	LogGC+CPR		LogGC+CPR (PAIOF)		HOLMES		HOLMES (PAIOF)		NoDoze		NoDoze (PAIOF)		DEPIMPACT		DEPIMPACT (PAIOF)	
	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.
Phishing Email	1.00	>0.01	1.00	>0.01(-48.9%)	1.00	0.07	1.00	0.05(-.02)	0.64	1.00	0.15(-.49)	0.33(-.66)	1.00	0.77	0.20(-.8)	0.03(-.74)
Drive-by Download	1.00	>0.01	1.00	>0.01(-32.8%)	1.00	0.07	1.00	0.05(-.02)	0.66	0.93	0.16(-.50)	0.25(-.68)	1.00	0.81	0.33(-.66)	0.07(-.74)
ShellShock	1.00	>0.01	1.00	>0.01(-51.9%)	1.00	0.10	1.00	0.08(-.02)	0.88	0.79	0.21(-.68)	0.25(-.75)	1.00	0.52	0.16(-.84)	0.23(-.29)
Trojan	1.00	>0.01	1.00	>0.01(-28.5%)	1.00	0.11	1.00	0.08(-.03)	0.91	0.64	0.22(-.69)	0.2(-.44)	1.00	0.83	0.31(-.69)	0.02(-.81)
Wget-gcc	1.00	>0.01	1.00	>0.01(-24.6%)	1.00	0.06	1.00	0.04(-.02)	0.50	1.00	0.08(-.42)	0.33(-.66)	1.00	0.66	0.09(-.91)	0.31(-.35)
Backdoor Download	1.00	>0.01	1.00	>0.01(-35.5%)	1.00	0.12	1.00	0.09(-.03)	1.00	1.00	0.18(-.82)	0.33(-.66)	1.00	0.80	0.11(-.89)	0.09(-.71)
Average	1.00	>0.01	1.00	>0.01(-87%)	1.00	0.09	1.00	0.065(-.025)	0.77	0.89	0.16(-.60)	0.3(-.59)	1.00	0.73	0.2(-.80)	0.125(-.69)

Table 5: PAIOF’s performance under White-box and Black box threat model.

Model	white-box (PAIOF)		black-box feature (PAIOF)		black-box real (PAIOF)		black-box (Random)	
	Recall	Precision	Recall	Precision	Recall	Precision	Recall	Precision
LogGC+CPR	1.00	0.01	1.00	0.01(+67%)	1.00	0.01(+87%)	1.00(+)	0.01(+87%)
HOLMES	1.00	0.01	1.00	0.01(+21%)	1.00	0.02(+.01)	1.00(+)	0.07(+.06)
NoDoze	0.12	0.30	0.22(+.10)	0.5(+.20)	0.27(+.15)	0.66(+.36)	0.56(+.44)	1.00(+.70)
DEPIMPACT	0.28	0.22	0.33(+.05)	0.26(+.04)	0.51(+.23)	0.34(+.12)	0.97(+.77)	0.69(+.56)
Average	0.60	0.14	0.64(+.04)	0.19(+.05)	0.67(+.07)	0.26(+.12)	0.88(+.28)	0.44(+.30)

THEIA, TRACE, and CADETS scenarios. The findings are outlined in Table 3. It is important to note that we employed the LogGC method in conjunction with the CPR method in our experiments, as they focus on entities that are either nodes or edges (both the graph compression-based method).

The results presented in Table 3 show that the graph compression-based scheme achieved the highest recall (100%) and the lowest precision (>0.01), despite reducing the majority of redundant nodes and edges. The rule-based scheme (HOLMES) further enhanced precision (3%) while maintaining high recall. In terms of priority-based schemes, the path-based Nodoe demonstrated excellent precision (100%) but only covered 48% of attacking nodes on average. On the other hand, the node-priority-based DEPIMPACT improved recall (90%) but saw a decrease in precision (52%) due to additional forward analysis.

Table 3 also provides the impact on the automated investigation tool after the addition of PAIOF, which we evaluated in our experiments with the number of added meta-behaviors n set to 1. Specifically, the priority-based schemes are significantly affected, on average, for the DEPIMPACT system, PAIOF reduces the node coverage by 62%, while its precision decreases to 30%. NoDoze’s recall decreases to 36%, and its precision decreases to 70%. For the knowledge-tag scheme HOLMES and the graph compression-based schemes LogGC and CPR, the goal of PAIOF is mainly to reduce their precision (the added obfuscated subgraphs do not hinder previous rule matching). For the HOLMES system, our scheme was able to reduce its precision by 81% on average, and by 87% for the graph compression schemes (CPR+LogGC). Overall, in the white-box evaluation scenarios, PAIOF demonstrates satisfactory performance. This enables attack investigation methods to produce erroneous results, leading to confusion and delayed response times for investigators.

4.3 Attacks Performance in Real-World Attack

To assess the impact of PAIOF in real-world attacks, we established six attack scenarios in Table 2 based on commonly used scenarios in previous studies [12, 19, 39, 51]. Following a similar assessment approach to the DARPA scenario, we compared the efficiency before and after implementing PAIOF. Experimental setups involved using one day’s audit logs to build a benign event frequency library for NoDoze and implementation attacks using the Linux Kali system. The obfuscated program generation was achieved by packaging a sequence of obfuscated behaviors using *pyinstaller*.

Table 4 presents the attack performance of PAIOF in different types of attack investigation schemes in real-world attack scenarios. PAIOF demonstrates greater sensitivity to prioritized scenarios, benefiting from the inclusion of more significant confusion subgraphs. On average, our scheme leads to a 60% decrease in recall and a 59% decrease in precision for the path-priority-based NoDoze, and an 80% decrease in recall and a 69% decrease in precision for the node-priority-based DEPIMPACT. In the case of the rule-based HOLMES, the addition of more rule-compliant nodes and edges results in 27% decrease in precision. The graph compression-based scheme experiences an 87% reduction in precision.

The impact of our scheme on the HOLMES system may not appear substantial in the feature space or real-world attacks, as most effects only show a slight decrease. This is due to our evaluation using the size of the HSG as the attack investigation results for HOLMES. The nodes we introduce in our scheme tend to have higher threat scores in the HSG, as they are matched to more advanced TTP rules such as DataLeak and Sensitive Read. By prioritizing nodes with higher threat scores in the HSG as the result of HOLMES, our scheme leads to a more significant decline in recall and precision. Furthermore, the graph compression-based

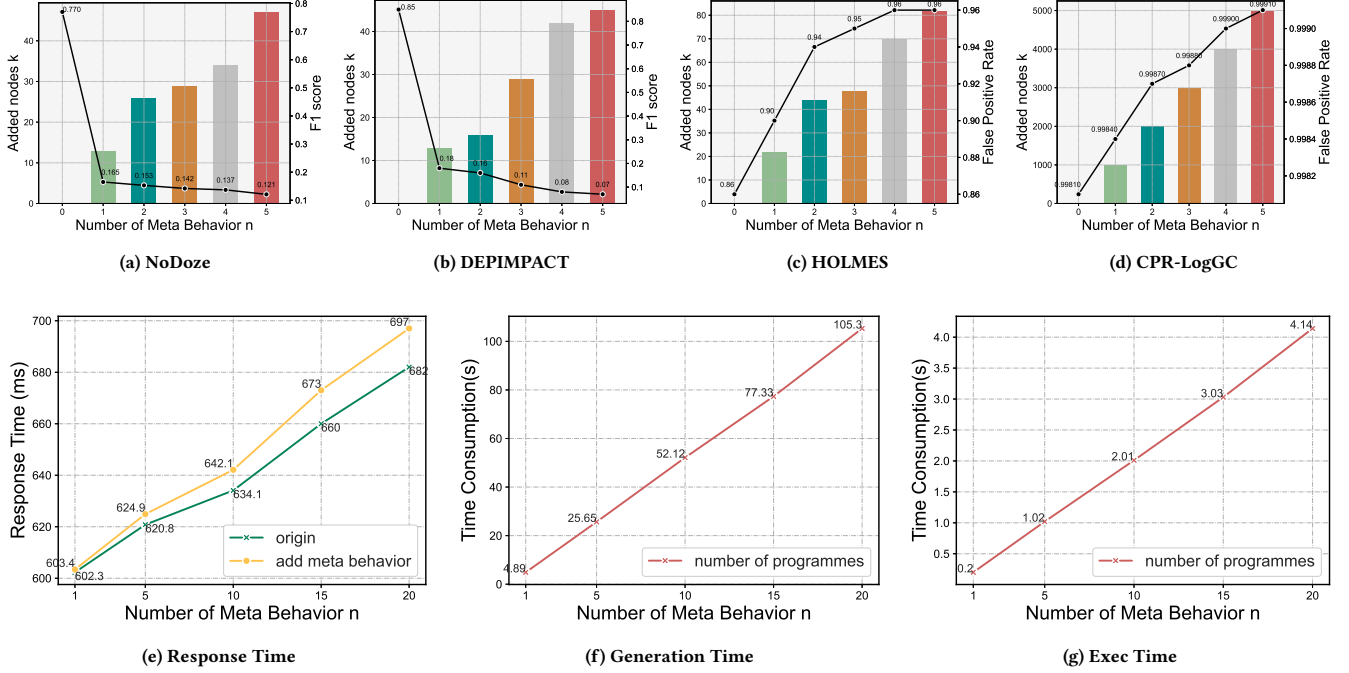


Figure 8: The number of meta-behaviors added versus the number of nodes added, F1 scores (NoDoze, DEPIMPACT), and FPR (HOLMES, CPR+LogGC). The line represents the change of F1 or FPR, and the bar chart represents the number of nodes added. The second line shows the impact of the increase in obfuscation on other indicators (response time, generation time, execution time).

scheme assumes that all nodes associated with a POI are attack nodes, increasing false positive rates and a decrease in precision.

4.4 Black-box Threat Models

In order to evaluate PAIOF in a more rigorous scenario, we additionally set up the threat model in black-box mode. This means that attackers do not have knowledge of the parameters and types used in the model. We aim to evaluate PAIOF’s performance comprehensively by incorporating all obfuscation goals (see Section 3.3) to the greatest extent possible. As a comparison, we also implemented a random node addition scenario (Random strategy) that does not consider any obfuscation goals. In the specific details of the evaluation, we control the number of nodes that add confounding subgraphs to be consistent to maintain the evaluation’s fairness for each type of approach. Additionally, we assigned equal weight to each obfuscated goal in the black-box model to maximize the achievability of evasion across multiple models.

A comparison of the effectiveness of PAIOF in different threat scenarios is presented in Table 5. The difference in recall and precision rates of PAIOF between white-box and black-box models is 4% and 5% (in feature space), respectively. In a real-world attack scenario, the difference of recall and precision increases to 7% and 12% in the black-box model. In comparison, the random strategy method showed higher variability in recall and precision, with an increase of 27% and 30%. This is due to the random addition of nodes not aligning with analytical key indicators, resulting in a

minimal impact on the final result. In general, in black-box threat models, multiple obfuscation goals are used to have an attack effect on as many methods as possible. Since the total number of added nodes remains the same, this leads to a slight decrease in the attack effectiveness of PAIOF. Nevertheless, PAIOF’s performance in the black-box scenario also almost reaches the performance in the white-box scenario (the performance decrease stays within 10%).

4.5 Impact of Practical Factors

This section provides an analysis of the practical applicability of PAIOF, examining its performance under various parameter settings. Additionally, we evaluate the impact of using PAIOF on the effectiveness of the investigation system, including the analysis of the implementation overhead necessitated by the obfuscation program.

Figure 8 provides the results of our experiments. It is important to note that we opted to use the FPR instead of the F1 score when evaluating the HOLMES and graph compression-based schemes. This decision was made due to the high recall and extremely low precision of this scheme, we focus primarily on the false positive rate resulting from the introduction of additional behavior. The findings indicate that for the NoDoze and DEPIMPACT scheme, PAIOF leads to an approximate 80% significant drop in F1 scores when only one meta-behavior is added, followed by a gradual (almost linear) decrease in the effect of increasing the number of meta-behaviors on F1 scores. PAIOF simulates numerous entity

interactions that meet the TTP rules, increasing the HOLMES system’s FPR from 86% to 96%. Nevertheless, the FPR stabilizes as the number of nodes increases. The graph compression-based scheme employs a conservative forensic analysis algorithm, yields high recall but extremely low precision. PAIOF adds more nodes and edges during meta-behavior construction, and the results demonstrate that our approach leads to a linear enhancement in the FPR of graph compression-based schemes.

While the incorporation of meta-behaviors has the potential to reduce the effectiveness of current provenance-based investigation schemes, it is crucial to recognize that these meta-behaviors originate from obfuscation programs. Therefore, the practicality of developing diverse obfuscation programs in real-world scenarios requires consideration, despite their notable efficacy. Our evaluation also examined the impact of integrating meta-behaviors on response times within provenance investigation schemes and on the performance of PAIOF. Initially, we assessed the impact of integrating meta-behaviors on response times within existing provenance-based investigation methodologies, followed by an evaluation of the associated costs involved in generating meta-behaviors (obfuscation programs) in real-world environments.

The results of our time consumption experiments are illustrated in Fig. 8(e), Fig. 8(f), and Fig. 8(g). Our approach to increasing the number of meta-behaviors demonstrates an improvement in the response time of the investigation tool. This improvement is attributed to PAIOF’s incorporation of nodes and edges for obfuscation, resulting in additional computational tasks such as weight computation, graph compression, and TTP matching. Furthermore, the construction time and execution time of obfuscation programs exhibit an almost linear relationship with the number of meta-behaviors. On average, it takes 2 seconds to construct an obfuscated program (including behavioral sequence generation and packing), and 0.2 seconds to execute an obfuscated program in a real-world attack scenario (blocking execution). For real-world applicability, an attacker can use PAIOF to create an obfuscation procedure before committing an attack. In fact, only the pre-prepared obfuscation programme needs to be executed during the attack, and this consumption (a few seconds) is acceptable for the attacker.

5 DISCUSSION

Defense against Evasion Attacks. While PAIOF primarily assesses existing provenance-based investigation schemes through the view of an attacker, the ultimate goal is to leverage PAIOF to pinpoint areas for enhancement and fortify the robustness and security of the current model. PAIOF generates meta-behavior by exposing evasion strategies embedded within prevalent investigation methodologies. For instance, investigation metrics for the prioritized approach are typically linked to node attributes in the graph, such as temporal correlation, node accessibility, and event frequency. Consequently, PAIOF thus creates more ‘obvious’ subgraphs in the graph with these investigation key indicators, enabling evasion attacks. It is apparent that these investigation indicators serve as the cornerstone of contemporary attack investigation systems, and safeguarding these metrics from unauthorized access by potential attackers represents a viable strategy against obfuscation attacks. Based on this insight, we propose some possible defense strategies:

- **Adopting dynamic key indicators.** The security of the investigation systems could be improved by adopting dynamic key indicators. This strategy will increase the attack difficulty, especially for long-term attacks, because dynamic key indicators can cause conceptual drift in the obfuscation program created by attackers.
- **Adopting more key indicators.** The investigation system can improve its adversarial robustness by incorporating more key indicators. Increasing the number of key indicators would require attackers to exert more effort in the obfuscation program to successfully execute attacks.
- **Adopting multiple investigation systems simultaneously.** Deploying multiple types of investigation systems on the host simultaneously can increase the likelihood of detecting attacks. This is because it becomes challenging for an attacker to identify the presence of various investigation systems at once.

While we have suggested three defense strategies, it is important to note that they are not without flaws. Implementing these strategies will likely increase the overhead of the investigation system. Introducing more key indicators may also result in a higher false positive rate. Since efficiency and robustness are often a trade-off in investigation systems, opting for a more robust approach will inevitably come with higher overheads.

Practical Applicability. The practical applicability of adversarial samples is a compelling topic, as most existing research focuses on feature space [32, 33, 41, 45, 57]. Adversarial samples generated in feature space can impact the target model, but are challenging to manipulate and reproduce in the problem space. Unlike image and malicious adversarial samples, in the context of provenance graph-based detection, adding nodes and edges solely in the feature space poses difficulties for attackers. This is because the provenance graph is derived from audit logs collected by the target host. Program behavior is logged in audit logs by system auditing frameworks like ETW [6] and Audit [3], from which the provenance graph is constructed by extracting the event ternary <src, dst, timestamp>. Therefore, constructing a practical and effective adversarial sample necessitates starting from the end-to-end program construction, which embodies the implementation logic of PAIOF.

In particular, PAIOF focuses on modeling the meta-behavior of the system to establish the connection between the system’s meta-behavior and the provenance graph. By identifying key indicators and constructing a sequence of behaviors, the obfuscation procedure is generated. This approach differs from previous schemes [13, 40] that directly incorporate node behavior into the provenance graph. The advantage of this method is the assurance of real-world applicability of the obfuscation program, as the meta-behavior sequences are derived from actual executable code segments, thereby reducing feature space and real-world evasion variability.

Applicability to Other Detectors. The transferability of obfuscation attacks is mainly reflected in the performance of existing attack examples on other models. Since PAIOF focuses on the generation of obfuscation attacks on three types of models in this paper, but in fact more and more recent research is extending new detection ideas, we still need to discuss the effectiveness of the generated obfuscation procedures for attacks on other models. Notably, whereas

previous adversarial attack examples operate at the feature space level [13, 40], PAIOF focuses predominantly on crafting attack instances applicable in real-world settings. Due to different goals, we do not directly compare the effects of the two in our experiments. Furthermore, we believe that PAIOF holds the potential to customize appropriate obfuscation goals for existing ML-based methods, thus achieving optimal reductions in recall and precision. This is because PAIOF adopts a template technology approach, and theoretically, all detection models possess what are known as detection ‘sensitive metrics’. This implies that our objective can be achieved by thoroughly analyzing key indicators and formulating corresponding obfuscation goals. Our future work on PAIOF will concentrate on exploring this avenue.

6 RELATED WORK

Causality Analysis via System Audit Logs. Causal analysis in provenance graph is commonly employed in various attack analyses [10, 21, 30]. Previous studies [23, 24] encountered dependency explosion issues when analyzing system causality, often assuming that each audit record relied on all preceding ones. To address this, researchers have explored techniques such as unit partitioning [27, 36, 37], modeling-based inference [25, 26], and application log fusion [17, 53]. While precise provenance presents a fundamental solution, concerns persist regarding high overhead and adaptability.

Additionally, researchers have also proposed alternative solutions that mitigate dependency explosion without relying on hardware modifications or additional software. Hossain et al. introduced SLEUTH [18] and MORSE [19], which employ tag flow and tag propagation techniques to facilitate the transfer of information between entities, resulting in a more streamlined dependency graph. PRI-OTRACKER [35] established a methodology for calculating the rarity of system entities as a measure of priority, thereby enhancing the efficiency of forensic analysis. NODOZE [16] expanded the concept of rarity to encompass entire paths, providing a more comprehensive understanding of system behavior. DEPIMPACT [12] determined edge weight scores based on diverse features such as time, data flow, and node degree. These anomaly scores were then aggregated along adjacent edges of the source graph, and these aggregated scores were subsequently utilized for alert classification. HOLMES [39] and RapSheet [15] designed TTPs rules tailored for graphs, leveraging the attack chain model. These rules enabled the extraction of more granular attack subgraphs for forensic analysis. In addition, some researchers designed some optimized storage schemes [20, 28, 46] for graphs to alleviate dependency explosion.

Recently, some new studies have also been presented by combining graph representation learning for attack investigation, which is different from the previous research [14, 48, 49] goal of anomaly detection through learning. DepComm [50] classifies different communities by performing community detection on provenance graphs. Then combines higher-order knowledge from HOLMES [39] to achieve attack investigation. ProGrapher [52] detects anomalies at the graph level, to support the forensic investigation, it ranks graph nodes based on their degrees of anomalousness. KAIROS [9] leverages deep graph learning and community discovery through causal dependencies in a provenance graph to detect anomalous

system behavior and correlate detected anomalies based on information flows between kernel objects.

Evasion Attacks for Provenance-based Detector. Wagner et al [47] first introduced the concept of mimicry attacks in 2002, which allow sophisticated attackers to hide their intrusions from IDS detection. They developed a theoretical framework to evaluate the security of IDS against mimicry attacks. This provided the basis for mimicry attacks to circumvent P-IDS (Provenance graph-based Intrusion Detection System) detectors. LI et al. [31] introduced an idea that for the first time questioned the robustness of the recently emerging provenance-based detectors, focusing on the potential risks of the dependency explosion problem addressed by existing P-IDS, they propose a mimicry attack-based circumvention method. Goyal et al. [13] demonstrated the first version of such evasion attacks in early 2023, which can persistently evade various P-IDS. Kunal et al. [40] improved on previous research and proposed the PROVNIJA framework, which reduces the number of new system events and expands the tolerable difference between the distribution of programs in the attacker’s dataset and the defender’s dataset. PAIOF expands on previous research by incorporating provenance-based investigation scenarios into the robustness against evasion attack analysis. Additionally, PAIOF creates a connection between the attacker’s operational behavior and provenance graphs, resulting in a more practical and straightforward obfuscation program to prove the current P-IDS needs to be extended and upgraded.

7 CONCLUSION

We introduce the PAIOF scheme, which is used to generate a practical obfuscation attack sample for existing provenance-based attack investigation methods. By defining obfuscation objectives through an analysis of critical metrics from established provenance-based forensic investigation methodologies. To ensure practical applicability, we establish an end-to-end model that correlates system meta-behavior with provenance graphs for obfuscation program generation. Experimental results reveal that PAIOF markedly diminishes the recall and precision rates of existing forensic investigation efforts, with reductions reaching up to 91% and averaging over 60%. Furthermore, our scheme has complete practical applicability.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers and the shepherd for their helpful suggestions. Their insightful comments significantly improved the quality of our paper. This research was supported in part by the National Key R&D Program of China (2023YFB3106900) and the National Natural Science Foundation of China (62072359, 62302362, 62072352).

REFERENCES

- [1] 2023. APT Notes. <https://github.com/kbandla/APTnotes>, Last accessed on 2023-12-25.
- [2] 2023. Common Vulnerability Scoring System v3.0: Specification Document. <https://www.first.org/cvss/specification-document>, Last accessed on 2023-11-21.
- [3] 2023. Linux Kernel Audit Subsystem. <https://github.com/linux-audit/audit>, Last accessed on 2023-11-21.
- [4] 2023. MANDIANT: Exposing One of China’s Cyber Espionage Units. <https://www.fireeye.com/content/dam/fireeye-www/services/pdfs/mandiant-apt1-report.pdf>, Last accessed on 2023-11-21.
- [5] 2023. Transparent Computing Engagement 3 DataRelease. <https://github.com/darpa-i2o/TransparentComputing/blob/master/README-E3.md>, Last accessed

- on 2023-12-25.
- [6] 2023. Windows ETW. <https://learn.microsoft.com/zh-cn/windowshardware/drivers/devtest/event-tracing-for-windows>, Last accessed on 2023-11-21.
 - [7] Abdullellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. ATLAS: A Sequence-Based Learning Approach for Attack Investigation. In *Proceedings of the USENIX Security Symposium*. 3005–3022.
 - [8] Adam Bates, Dave Jing Tian, Kevin RB Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *Proceedings of the 24th USENIX Security Symposium (USENIX Security 15)*. 319–334.
 - [9] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2023. Kairos: Practical Intrusion Detection and Investigation Using Whole-System Provenance. *arXiv preprint arXiv:2308.05034* (2023).
 - [10] Feng Dong, Shaofei Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, et al. 2023. Are We There Yet? An Industrial Viewpoint on Provenance-Based Endpoint Detection and Response Tools. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2396–2410.
 - [11] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. DISTDET: A Cost-Effective Distributed Cyber Threat Detection System. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*.
 - [12] Pengcheng Fang, Peng Gao, Changlin Liu, Erman Ayday, Kangkook Jee, Ting Wang, Yanfang Fanny Ye, Zhuotao Liu, and Xusheng Xiao. 2022. Back-Propagating System Dependency Impact for Attack Investigation. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*. 2461–2478.
 - [13] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. 2023. Sometimes, You Aren't What You Do: Mimicry Attacks Against Provenance Graph Host Intrusion Detection Systems. In *Proceedings of the 30th Network and Distributed System Security Symposium (NDSS)*.
 - [14] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. 2020. Unicorn: Runtime Provenance-Based Detector for Advanced Persistent Threats. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
 - [15] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical Provenance Analysis for Endpoint Detection and Response Systems. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1172–1189.
 - [16] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. Nodose: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*.
 - [17] Wajih Ul Hassan, Mohammad Ali Nouredine, Pubali Datta, and Adam Bates. 2020. OmegaLog: High-Fidelity Attack Investigation via Transparent Multi-Layer Log Analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
 - [18] Md Nahid Hossain, Sadegh M Milajerdi, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R Sekar, Scott D Stoller, and V.N. Venkatakrishnan. 2017. SLEUTH: Real-Time Attack Scenario Reconstruction from COTS Audit Data. In *Proceedings of the USENIX Security Symposium*. 487–504.
 - [19] Md Nahid Hossain, Sanaz Sheikh, and R Sekar. 2020. Combating Dependence Explosion in Forensic Analysis Using Alternative Tag Propagation Semantics. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1139–1155.
 - [20] Md Nahid Hossain, Junao Wang, R Sekar, and Scott D Stoller. 2018. Dependence-Preserving Data Compaction for Scalable Forensic Analysis. In *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. 1723–1740.
 - [21] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2022. SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 307–325.
 - [22] Peng Jiang, Jifan Xiao, Ding Li, Hongyi Yu, Yu Bai, Yao Guo, and Xiangqun Chen. 2023. Detecting Malicious Websites from the Perspective of System Provenance Analysis. *IEEE Transactions on Dependable and Secure Computing* (2023).
 - [23] Samuel T King and Peter M Chen. 2003. Backtracking Intrusions. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles (SOSP)*. 223–236.
 - [24] Samuel T King, Zhuoqing Morley Mao, Dominic G Lucchetti, and Peter M Chen. 2005. Enriching Intrusion Alerts Through Multi-Host Causality. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Citeseer.
 - [25] Yonghwi Kwon, Dohyeong Kim, William Nick Sumner, Kyungtae Kim, Brendan Saltaformaggio, Xiangyu Zhang, and Dongyan Xu. 2016. LDX: Causality Inference by Lightweight Dual Execution. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 503–515.
 - [26] Yonghwi Kwon, Fei Wang, Weihang Wang, Kyu Hyung Lee, Wen-Chuan Lee, Shiqing Ma, Xiangyu Zhang, Dongyan Xu, Somesh Jha, Gabriela F Ciocarlie, et al. 2018. MCI: Modeling-Based Causality Inference in Audit Logging for Attack Investigation. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Vol. 2. 4.
 - [27] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. High Accuracy Attack Provenance via Binary-based Execution Partition. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Vol. 16.
 - [28] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. LogGC: Garbage Collecting Audit Log. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1005–1016.
 - [29] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2023. NODLINK: An Online System for Fine-Grained APT Attack Detection and Investigation. *arXiv preprint arXiv:2311.02331* (2023).
 - [30] Zhenyuan Li, Qi Alfred Chen, Runqing Yang, Yan Chen, and Wei Ruan. 2021. Threat Detection and Investigation with System-Level Provenance Graphs: A Survey. *Computers & Security* 106 (2021), 102282.
 - [31] Zhenyuan Li, Runqing Yang, Qi Alfred Chen, and Yan Chen. 2020. Mimic the Whole Attack Chain: A First Look at Evasion Against Provenance Graph Based Detection. In *Annual Computer Security Applications Conference (ACSAC)*.
 - [32] Xiang Ling, Lingfei Wu, Xiangyu Zhang, Zhenqing Qu, Wei Deng, Xiang Chen, Yaguan Qian, Chunming Wu, Shouling Ji, Tianyue Luo, et al. 2023. Adversarial Attacks Against Windows PE Malware Detection: A Survey of the State-of-the-Art. *Computers & Security* 128 (2023), 103134.
 - [33] Xinbo Liu, Jiliang Zhang, Yaping Lin, and He Li. 2019. ATPMA: Attacking Machine Learning-Based Malware Visualization Detection Methods via Adversarial Examples. In *Proceedings of the International Symposium on Quality of Service (IWQoS)*. 1–10.
 - [34] Yushan Liu, Xiaokui Shu, Yixin Sun, Jiyong Jang, and Prateek Mittal. 2022. RAPID: Real-Time Alert Investigation with Context-Aware Prioritization for Efficient Threat Discovery. In *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC)*. 827–840.
 - [35] Yushan Liu, Mu Zhang, Ding Li, Kangkook Jee, Zhichun Li, Zhenyu Wu, Junghwan Rhee, and Prateek Mittal. 2018. Towards a Timely Causality Analysis for Enterprise Security. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
 - [36] Shiqing Ma, Juan Zhai, Fei Wang, Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2017. MPI: Multiple Perspective Attack Investigation with Semantic Aware Execution Partitioning. In *Proceedings of the USENIX Security Symposium*. 1111–1128.
 - [37] Shiqing Ma, Xiangyu Zhang, Dongyan Xu, et al. 2016. Protracer: Towards Practical Provenance Tracing by Alternating Between Logging and Tainting. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Vol. 2. 4.
 - [38] Sadegh M Milajerdi, Birhanu Eshete, Rigel Gjomemo, and V.N. Venkatakrishnan. 2019. Poirot: Aligning Attack Behavior with Kernel Audit Records for Cyber Threat Hunting. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1795–1812.
 - [39] Sadegh M Milajerdi, Rigel Gjomemo, Birhanu Eshete, Ramachandran Sekar, and V.N. Venkatakrishnan. 2019. Holmes: Real-Time APT Detection through Correlation of Suspicious Information Flows. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1137–1152.
 - [40] Kunal Mukherjee, Joshua Wiedemeier, Tianhao Wang, James Wei, Feng Chen, Muhyun Kim, Murat Kantarcioglu, and Kangkook Jee. 2023. Evading Provenance-Based ML Detectors with Adversarial System Actions. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 1199–1216.
 - [41] Kien Nguyen, Tharindu Fernando, Clinton Fookes, and Sridha Sridharan. 2023. Physical Adversarial Attacks for Surveillance: A Survey. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
 - [42] Riccardo Paccagnella, Pubali Datta, Wajih Ul Hassan, Adam Bates, Christopher Fletcher, Andrew Miller, and Dave Tian. 2020. Custos: Practical Tamper-Evident Auditing of Operating Systems Using Trusted Execution. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
 - [43] Riccardo Paccagnella, Kevin Liao, Dave Tian, and Adam Bates. 2020. Logging to the Danger Zone: Race Condition Attacks and Defenses on System Audit Frameworks. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1551–1574.
 - [44] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eysers, Margo Seltzer, and Jean Bacon. 2017. Practical Whole-System Provenance Capture. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC)*. 405–418.
 - [45] Ishai Rosenberg, Asaf Shabtai, Yuval Elovici, and Lior Rokach. 2020. Query-Efficient Black-Box Attack Against Sequence-Based Malware Classifiers. In *Proceedings of the 36th Annual Computer Security Applications Conference (ACSAC)*. 611–626.
 - [46] Yutao Tang, Ding Li, Zhichun Li, Mu Zhang, Kangkook Jee, Xusheng Xiao, Zhenyu Wu, Junghwan Rhee, Fengyuan Xu, and Qun Li. 2018. Nodemerge: Template Based Efficient Data Reduction for Big-Data Causality Analysis. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1324–1337.

- [47] David Wagner and Paolo Soto. 2002. Mimicry Attacks on Host-Based Intrusion Detection Systems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*. 255–264.
- [48] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A Gunter, et al. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*.
- [49] Yulai Xie, Yafeng Wu, Dan Feng, and Darrell Long. 2019. P-Gaussian: Provenance-Based Gaussian Distribution for Detecting Intrusion Behavior Variants Using High Efficient and Real-Time Memory Databases. *IEEE Transactions on Dependable and Secure Computing* 18, 6 (2019), 2658–2674.
- [50] Zhiqiang Xu, Pengcheng Fang, Changlin Liu, Xusheng Xiao, Yu Wen, and Dan Meng. 2022. DepComm: Graph Summarization on System Audit Logs for Attack Investigation. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 540–557.
- [51] Zhang Xu, Zhenyu Wu, Zhichun Li, Kangkook Jee, Junghwan Rhee, Xusheng Xiao, Fengyuan Xu, Haining Wang, and Guofei Jiang. 2016. High Fidelity Data Reduction for Big Data Security Dependency Analyses. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 504–516.
- [52] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PROGRAPHER: An Anomaly Detection System Based on Provenance Graph Embedding. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*. 4355–4372.
- [53] Runqing Yang, Shiqing Ma, Haitao Xu, Xiangyu Zhang, and Yan Chen. 2020. UIScope: Accurate, Instrumentation-Free, and Visible Attack Investigation for GUI Applications. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- [54] Jun Zeng, Zheng Leong Chua, Yinfang Chen, Kaihang Ji, Zhenkai Liang, and Jian Mao. 2021. WATSON: Abstracting Behaviors from Audit Logs via Aggregation of Contextual Semantics. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- [55] Jun Zeng, Chuqi Zhang, and Zhenkai Liang. 2022. PalanTir: Optimizing Attack Provenance with Hardware-Enhanced System Observability. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 3135–3149.
- [56] Jun Zengy, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. 2022. Shadewatcher: Recommendation-Guided Cyber Threat Analysis Using System Audit Records. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 489–506.
- [57] Fangtian Zhong, Xiuzhen Cheng, Dongxiao Yu, Bei Gong, Shuaiwen Song, and Jiguo Yu. 2023. MalFox: Camouflaged Adversarial Malware Example Generation Based on Conv-GANs Against Black-Box Detectors. *IEEE Trans. Comput.* (2023).
- [58] Tiantian Zhu, Jinkai Yu, Chunlin Xiong, Wenrui Cheng, Qixuan Yuan, Jie Ying, Tieming Chen, Jiabo Zhang, Mingqi Lv, Yan Chen, et al. 2023. APTSHIELD: A Stable, Efficient and Real-Time APT Detection System for Linux Hosts. *IEEE Transactions on Dependable and Secure Computing* (2023).